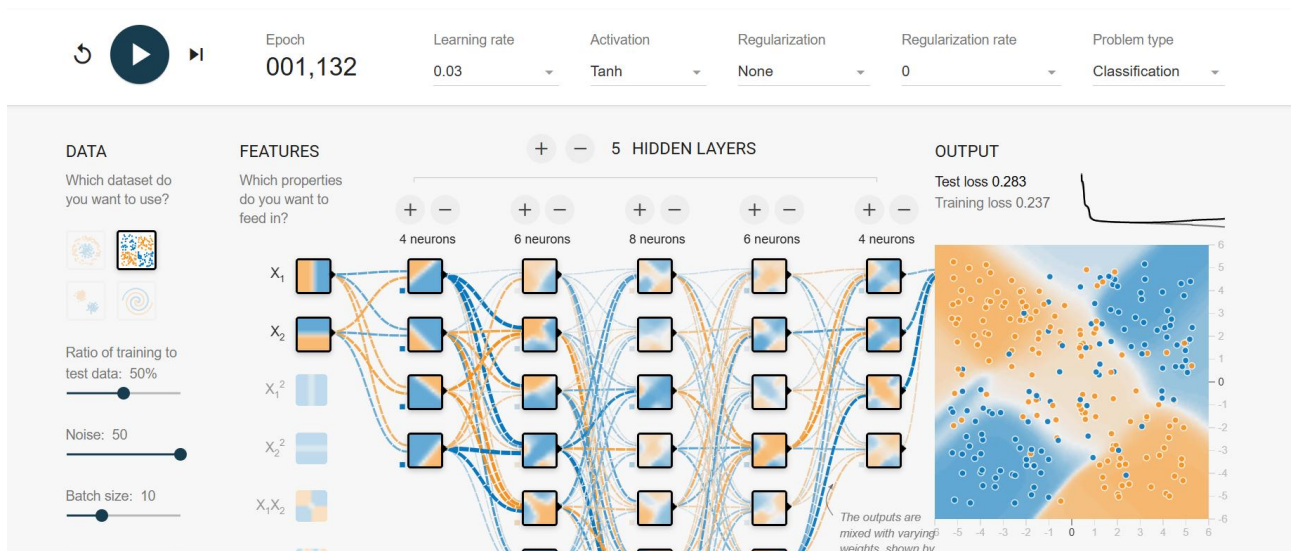
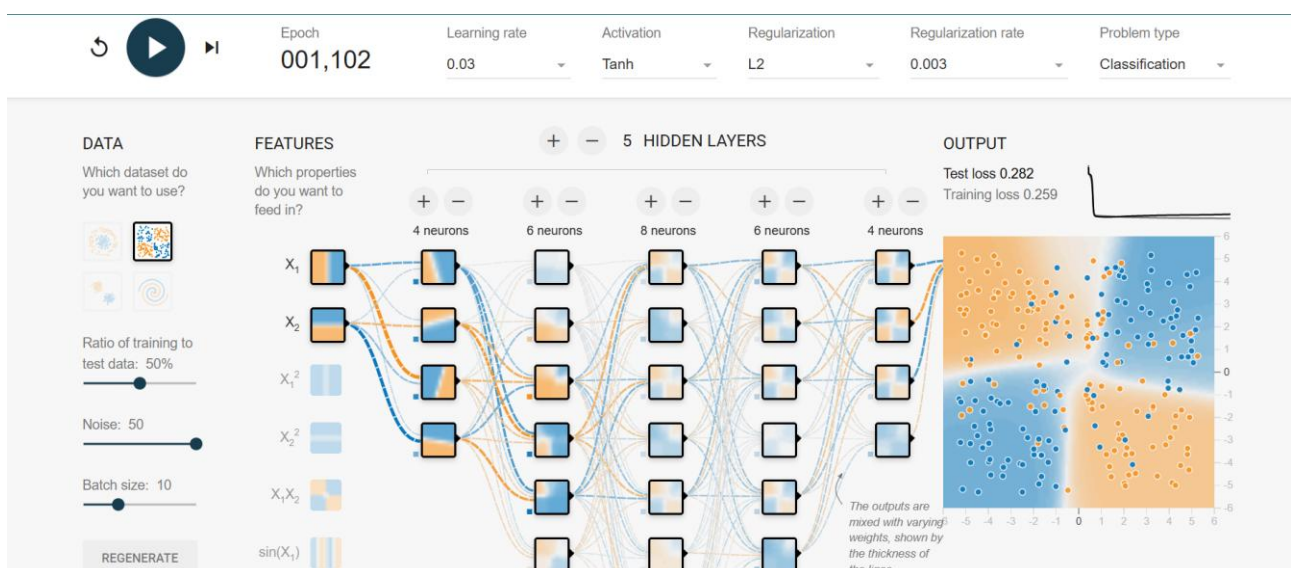


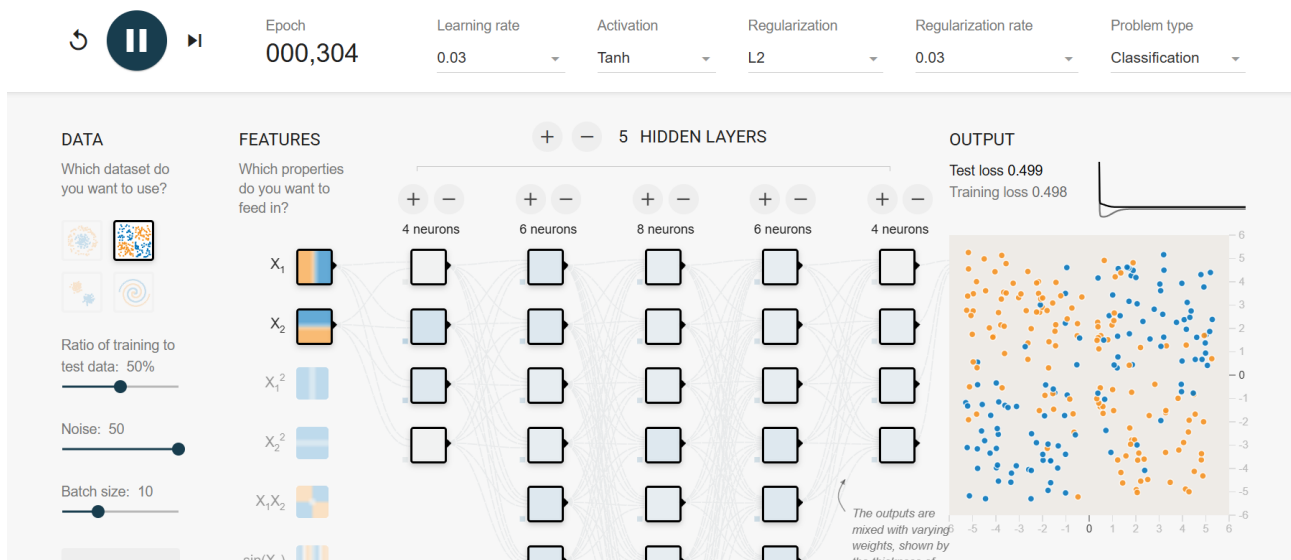
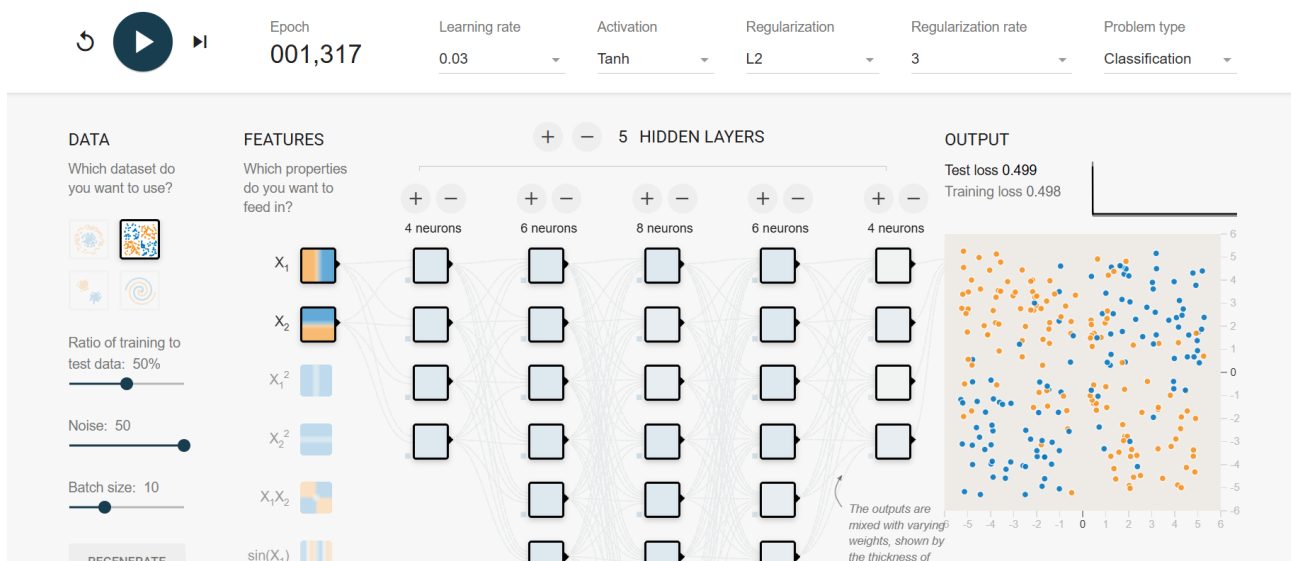
Tensorflow Playground Regularization Example



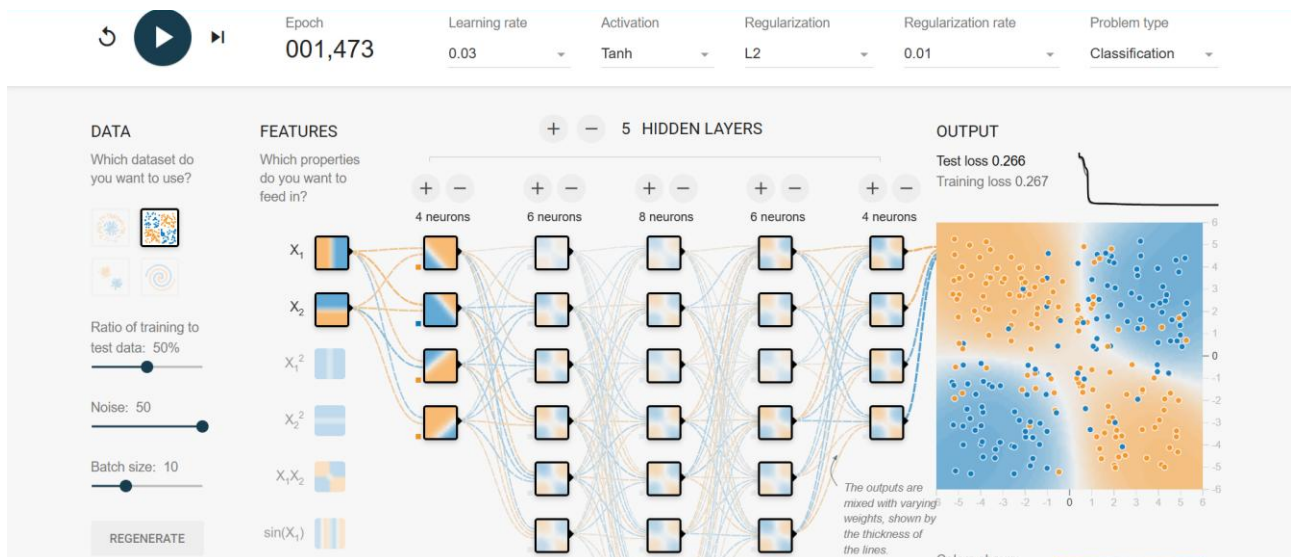
For this first run, the network has a deep and wide architecture, tanh is the activation function, we have 50% noise, a batch size of 10, and no regularization. After running for over 1000 epochs, we observe a classic case of overfitting. Because the network has many neurons and layers, it memorizes the training data, including the random 50% noise. This results in highly complex, with irregular decision boundaries. Looking at the learning curve, the training loss continues to decrease, but the test loss eventually stabilizes and then starts to diverge (increase). This growing generalization gap indicates that the model is failing to generalize to unseen data. We don't want this. Maybe with regularization it gets better.



Introducing a low L2 regularization rate starts to penalize large weights. The generalization gap becomes smaller, and the decision boundaries begin to smooth out. However, with a very small rate, some overtraining is still visible, meaning the network is still capturing some of the noise.

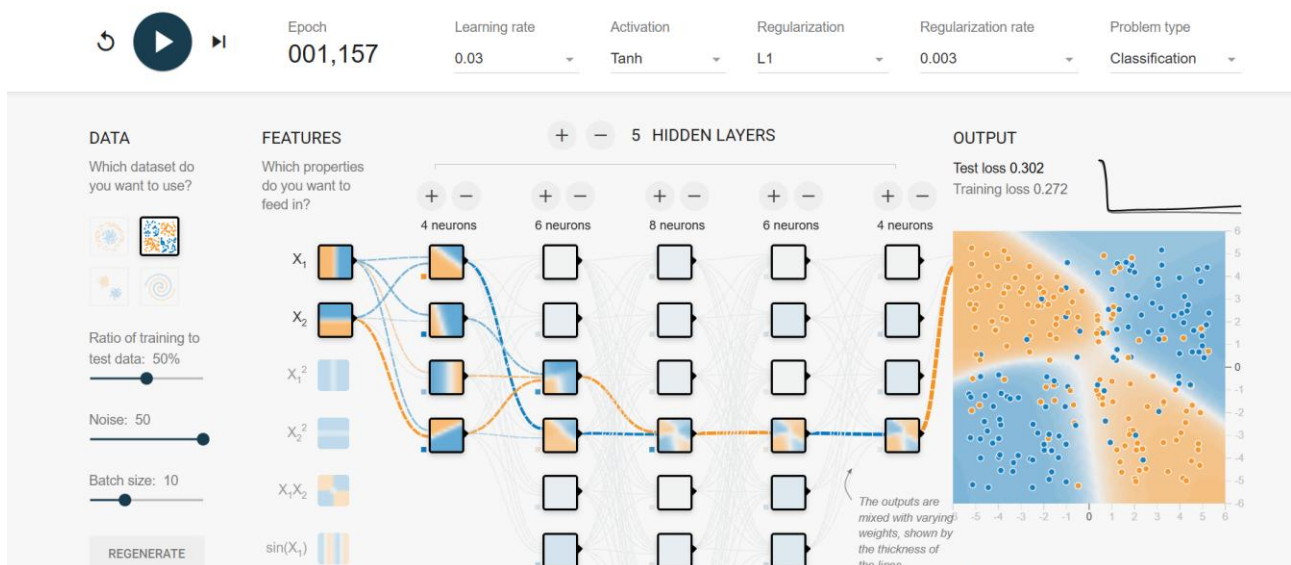


When testing a high L2 regularization rate (0.03, the same as the learning rate, and even higher), the network completely fails to learn. The penalty applied to the weights is so aggressive that the network is forced to keep its weights near zero, preventing it from forming the complex combinations needed to separate the checkerboard pattern. The loss curve remains flat.

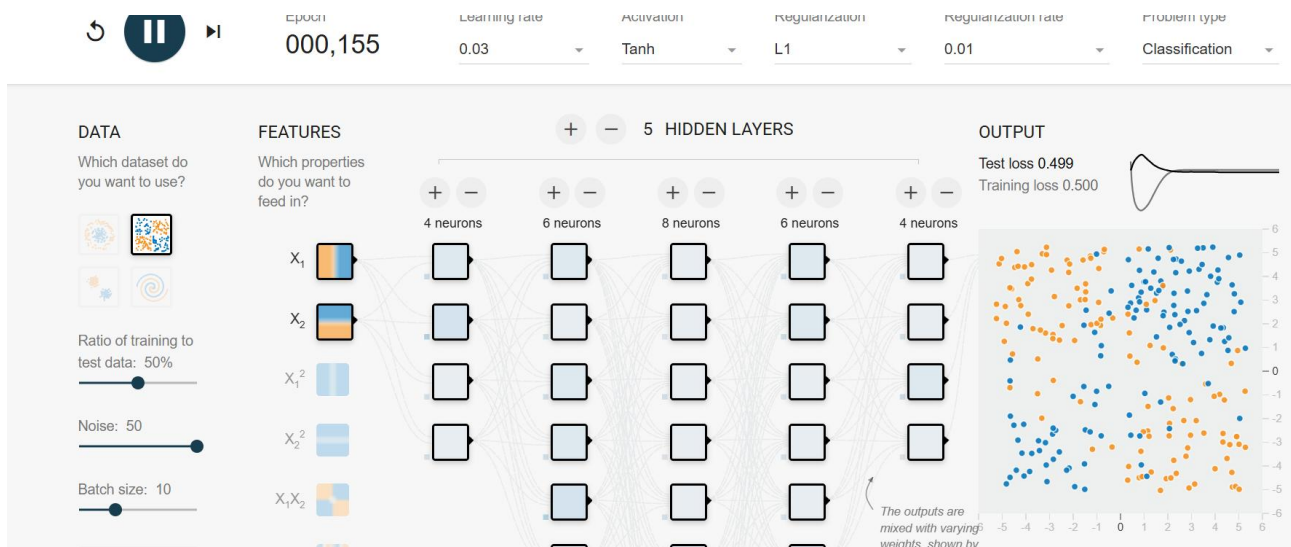


A more balanced rate of 0.01 provides the best results. The regularization perfectly counteracts the network's tendency to overfit. The decision boundaries are smooth, capturing the underlying checkerboard distribution rather than the noisy points. The training and test loss curves converge closely without diverging, proving that the overfitting problem has been successfully solved.

Now we compare with the L1 normalization:



Switching to L1 regularization (at a rate of 0.003) yields a test loss similar to the optimal L2 configuration, but the underlying mechanism is drastically different. The main difference is in the weights: we see that instead of decreasing all weights (in L2 regularization), here the strategy is to basically let decay the neurons that don't help the classification, and they become essentially zero. This effectively "disconnects" certain neurons leaving only the most important paths active.



This last example shows that L1 regularization is generally more aggressive than L2. When applying a rate of 0.01 (which worked perfectly for L2), the L1 penalty is too harsh. It drives too many weights to zero prematurely, effectively killing the network's ability to start learning and minimizing the loss. This demonstrates that L1 requires careful tuning, often with lower rates than L2, to avoid stopping the network's functions.